Introduction à jQuery



PRÉSENTATION DE JQUERY

- Bibliothèque JavaScript libre
- Permet de traverser et de manipuler facilement l'arbre DOM des pages web
- Permet par exemple de changer/ajouter une classe CSS, créer des animations, modifier des attributs, etc.
- Gère les événements JavaScript
- Fait des requêtes AJAX plus simplement

PRÉSENTATION DE JQUERY

- jQuery a été créée par John Resig en 2006
- La version la plus récente est : 4.0.0

POURQUOI UTILISER JQUERY

- Facile à apprendre (utilise la syntaxe CSS pour la sélection)
- De petite taille
- Documentation fournie et support disponible
- Compatibilité multi-navigateurs (cross-browser)
- Très utilisé

INSTALLATION

- jQuery est tout simplement un fichier JavaScript
  - Il vous suffit donc de le télécharger sur le site officiel : **https://jquery.com/**
- Intégration dans la page Web

```
<script src="js/jquery-4.0.0.min.js"></script>
```

- Ou directement en ligne

```
<script src="https://code.jquery.com/jquery-4.0.0.min.js"
integrity="sha256-IVpPzhjoR2NZIyL43mC6YQjV/01P11hS3J4h1Wn1LqU="
crossorigin="anonymous"></script>
```

```
<!doctype html>
<html>
<head>
        <meta charset="utf-8">
        <title>JQuery</title>
</head>
<body>
        .....

        <script          src="js/jquery-4.0.0.min.js" ></script>
        <script>         //Code javascript    </script>
        <script src=""></script>
</body>
</html>
```

DÉMARRAGE

- Pour être sûr que jQuery puisse fonctionner et trouver les éléments qu'on lui demande, le DOM (les éléments) doit être prêt.
- Pour cela on utilise la fonction **ready()** : elle se lance à l'instant où le DOM est totalement chargé.

```
$(document).ready(function(){
        // Votre code jQuery ici
});
```

```
$(function() {
        // Votre code jQuery ici
});
```

EXEMPLE

```
$("div").addClass("special");
```

- **$** est l'objet jQuery (nommé aussi **jQuery**).
- **"div"** recherche les éléments **div** du document
- **$("div")** retourne un ensemble jQuery (contenant o ou plusieurs nœuds DOM).
- **addClass(…)** une fonction qui permet de modifier tous les div (dans ce cas ajouter une classe **special**)

PRINCIPE DE BASE

- Principe de base de jQuery
  - Sélectionner des éléments
  - Et en faire quelque chose

LES SÉLECTEURS

- jQuery utilise les sélecteurs CSS
  - Le sélecteur d'éléments : $("p"), $("a"), $("div"), …
  - Le sélecteur d'identifiant #id : $("#test")
  - Le sélecteur de classes .class : $(".test")
- En plus de quelques sélecteurs spécifiques :
  - (:first, :last, :has(), :visible, :hidden, :selected, etc)

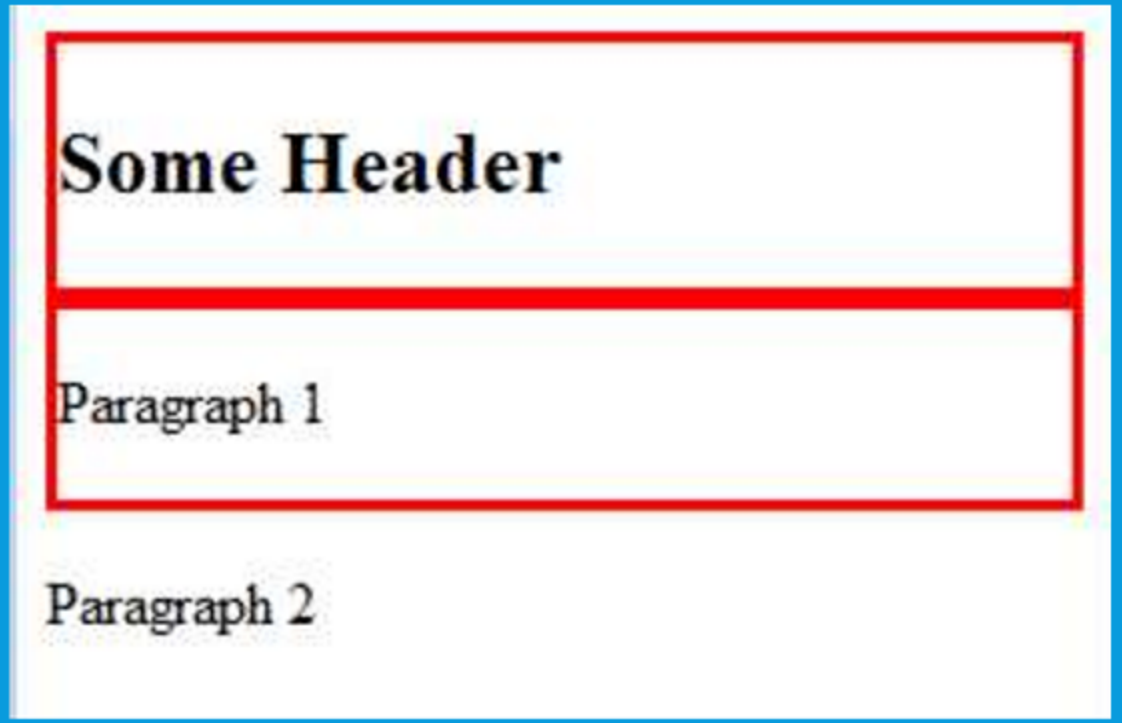
EXEMPLE

```
$("div").css("border", "3px solid red");
```

```
<div id="body">
    <h2>Some Header</h2>
</div>

<div class="contents">
        <p>Paragraph 1</p>
</div>

<p>Paragraph 2</p>
```



LES MÉTHODES

- Les méthodes jQuery peuvent être divisées en plusieurs catégories :
  - Les effets
  - Les méthodes de parcours du DOM
  - Les méthodes de taille et de position
  - Les méthodes de manipulation
  - Les événements
  - La méthode **each()**
  - La méthode **animate()**

LES EFFETS

- Avec jQuery il est possible de déclencher des effets prédéfinis sur les objets sélectionnés

Effet	Description
show()	affiche le(s) élément(s) sélectionné(s)
hide()	masque le(s) élément(s) sélectionné(s)
toggle()	affiche ou masque le(s) élément(s) sélectionné(s) en fonction de son/leur état actuel
fadeIn()	fait apparaître en fondu le(s) élément(s) sélectionné(s)
fadeOut()	fait disparaître en fondu le(s) élément(s) sélectionné(s)
fadeTo()	amène progressivement le(s) élément(s) sélectionné(s) à une opacité donnée
slideUp()	masque le(s) élément(s) sélectionné(s) avec un défilement vers le haut
slideDown()	affichele(s) élément(s) sélectionné(s) avec un défilement vers le bas
slideToggle()	aoffiche ou masque le(s) élément(s) sélectionné(s) avec un défilement en fonction de son/leur état actuel

EXEMPLE

```
$("div").hide();
```

```
$("div").hide(3000);
```

```
$("div").show();
```

```
$("div").toggle();
```

```
$("div").show(3000).hide(3000);
```

CHAINING (CHAINAGE)

- Vous pouvez effectuer plusieurs actions sur un même ensemble d'éléments

```
$("div").hide();
$("div").css("color","blue");
$("div").show();
```

```
$("div").hide().css("color","blue").show();
```

TRAVERSING (PARCOURIR LE DOM)

- On parle de parcours ou de navigation
- Parfois les sélecteurs CSS ne suffisent pas
- jQuery fournit un ensemble de fonctions qui permettent de parcourir l'arborescence du DOM
  - (parent(), next(), prev(), children(), find(), etc)

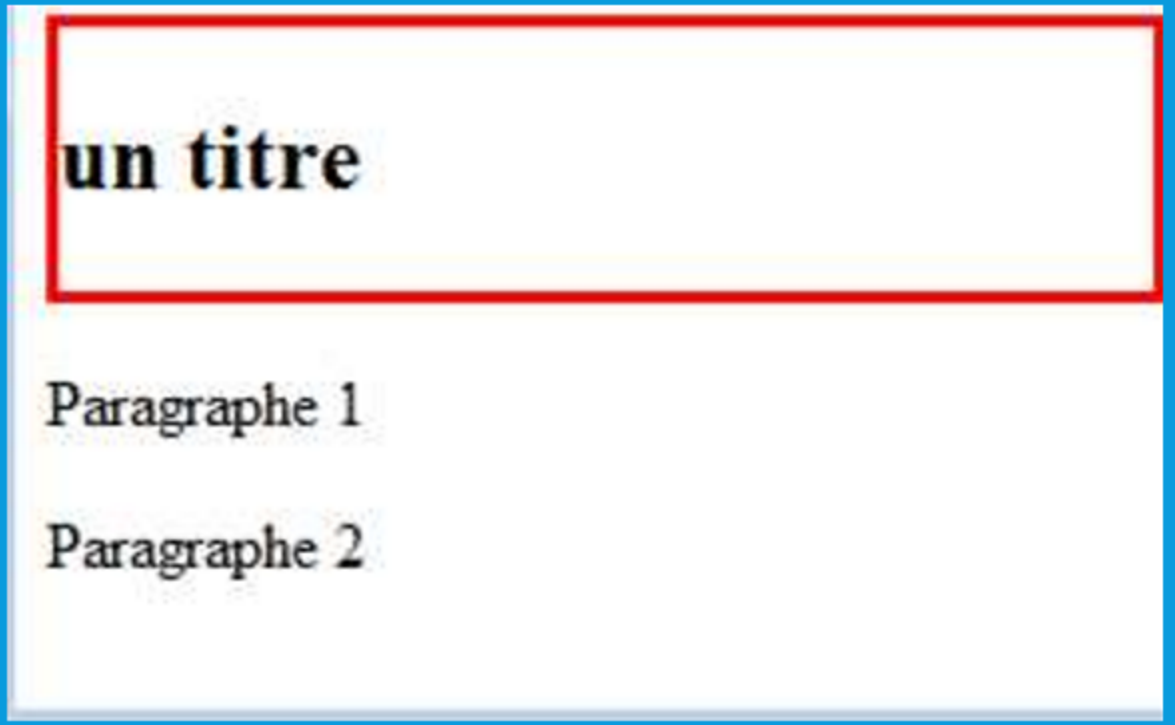
EXEMPLE

```
$("h2").parent().css("border", "3px solid red");
```

```
<div id="body">
    <h2>un titre</h2>
</div>

<div class="contents">
        <p>Paragraphe 1</p>
</div>

<p>Paragraphe 2</p>
```



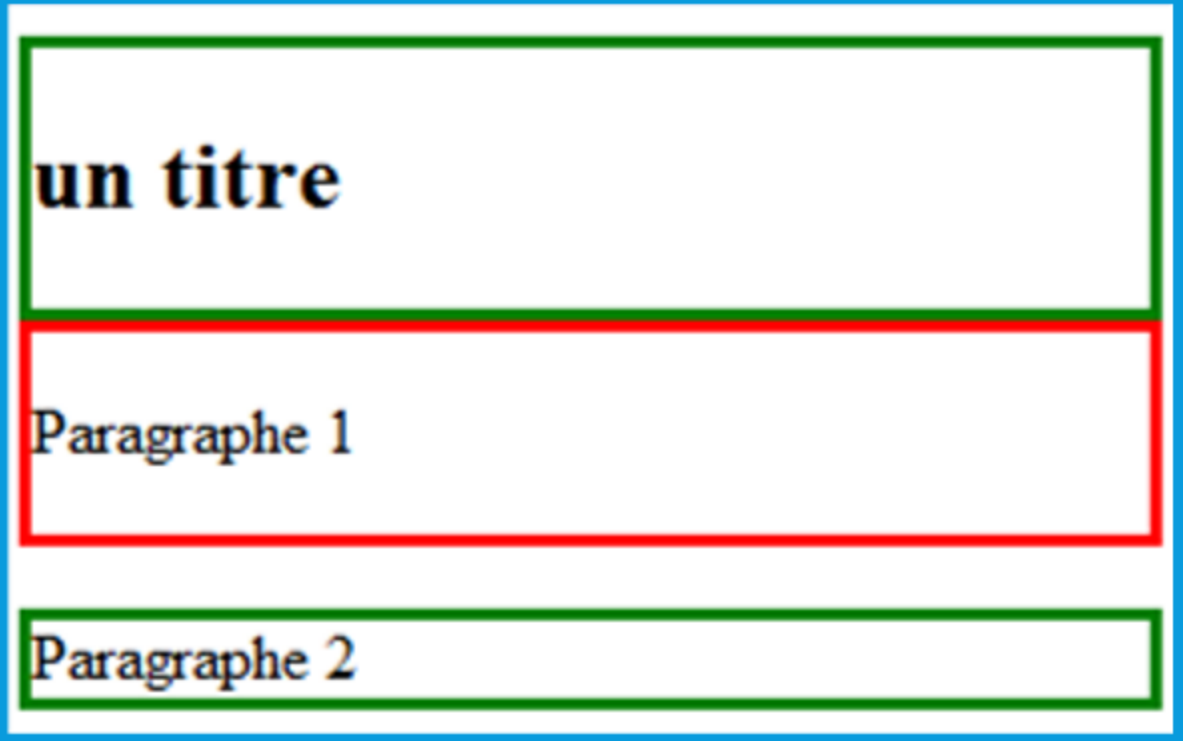
EXEMPLE

```
$(".para")
        .parent().css(”border", ”3px solid red")
        .siblings().css(”border", ”3px solid green");
```

```
<div id="body">
    <h2>Some Header</h2>
</div>

<div class="contents">
 <p class="para">Paragraphe 1</p>
</div>

<p>Paragraphe 2</p>
```



TAILLE ET POSITION

- Les méthodes jQuery dédiées aux dimensions et à la position permettent d'ajuster les proportions et la disposition des éléments

Méthode	Description
height()	hauteur d'un cadre sans les marges intérieures, extérieurs ni les bords
width()	largeur d'un cadre sans les marges intérieures, extérieurs ni les bords
innerHeight()	hauteur en comptant les marges intérieures
innerWidth()	largeur en comptant les marges intérieures
offset()	définit les coordonnées d'un élément relativement au coin en haut à gauche de l'objet document
position()	définit les coordonnées d'un élément relativement à son parent direct
.....	

MANIPULER LES CONTENUS

- Après la sélection des éléments on peut leurs appliquer plusieurs actions :
  - Manipuler les **attributs** (addClass, att, …)
  - Manipuler les **feuilles de styles CSS** (css)
  - Manipulation du **DOM** (append, prepend, remove, clone, …)

MANIPULER LES CONTENUS

Méthode	Description
html()	remplace le contenu HTML de la page
text()	remplace le texte de la page
replaceWith()	remplace le(s) élément(s) sélectionné(s) entièrement, pas juste le texte ou le HTML à l'intérieur
remove()	supprime les éléments de la page
before()	insère du contenu avant le(s) élément(s) sélectionné(s)
after()	insère du contenu après le(s) élément(s) sélectionné(s)
prepend()	insère du contenu à l'intérieur du/des élément(s) sélectionné(s) (après la balise HTML ouvrante)
append()	insère du contenu à l'intérieur du/des élément(s) sélectionné(s) (avant la balise HTML fermante)
attr()	définit un attribut et sa valeur ou simplement récupère sa valeur
.....	

EXERCICE 1

```
<ul>
    <li><a href="http://google.com/" target="_blank">Google</a></li>
    <li><a href="http://microsoft.com/">Microsoft</a></li>
    <li><a href="http://yahoo.com/" target="_blank">Yahoo</a></li>
</ul>
```



MANIPULATION (.CSS())

```
$("a[target=_blank]")
  .css({
        color : "red",
        fontWeight : "bold"
});
```





MANIPULATION (.AFTER)

```
$("a[target=_blank]")
  .after("<img src='open.png'/>");
```





MANIPULATION (.APPEND)

```
$("a[target=_blank]")
  .append(" (S'ouvre dans une autre fenêtre)");
```

- Google
- Microsoft
- Yahoo

- Google (S'ouvre dans autre fenêtre)
- Microsoft
- Yahoo (S'ouvre dans autre fenêtre)

GÉRER LES ÉVÈNEMENTS

- jQuery vous propose un ensemble de méthodes permettant la gestion des évènements

.click()	.keydown()
.scroll()	.keyup()
.hover()	.keypress()
.mouseover()	.focus()
.mouseout()	.blur()
.mouseenter()	.resize()
.mouseleave()	…

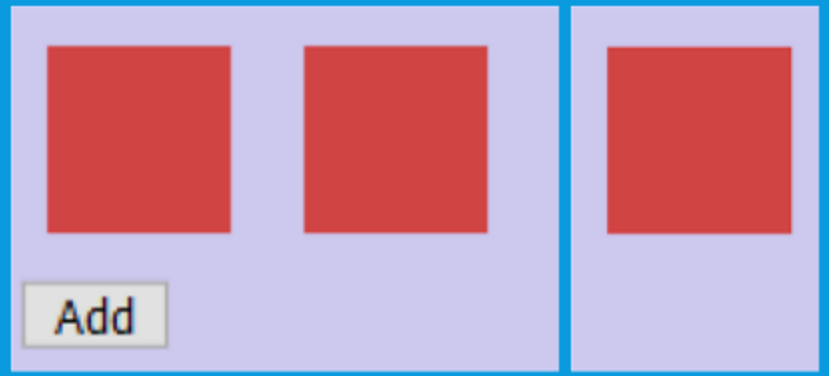
GÉRER LES ÉVÈNEMENTS

- Ou la méthode jQuery **on()**
- Elle permet de créer ce qu'on appelle "un écouteur d'événement" ("event listener" en anglais)

```
.on('click', function() { ... }
.on('scroll', function() { ... }
.on('mouseover', function() { ... }
.on('mouseenter', function() { ... }
.on('mouseleave', function() { ... }
.on('keydown', function() { ... }
.on('keyup', function() { ... }
.on('keypress', function() { ... }
.on('focus', function() { ... }
.....
```

UN CAS PARTICULIER

```
<div id="target">
    <div class="box"></div>
    <div class="box"></div>
</div>
<div>
    <button id="add">Add</button>
</div>
```




```
$('#add').on('click', function(){
        $('#target').append('<div class="box"></div>');
})
```




```
$('.box').click(function () {    // $('.box').on('click', function(){
        $(this).css('border', '3px solid white');
})
```

UN CAS PARTICULIER

```
$('.box').click(function () {    // $('.box').on('click', function(){
$(document).on('click', '.box', function(){
        $(this).css('border', '3px solid white');
})
```



EFFET (.HIDE()/.SHOW())

```
$("div.block").hide("slow", function(){
  $(this).show("slow");
});
```

- Que fait ce code ?

LA MÉTHODE .EACH()

- La méthode **.each()** indique une fonction à exécuter pour chaque élément correspondant.

```
$(selector).each(function(index, element))
```

```
$("p").each(function(index, element) {
    console.log(element.innerText);
    //console.log($(this).text());
});
```

LA MÉTHODE JQUERY.EACH()

- La méthode **$.each()** indique une fonction qui permet de parcourir des objets et des Arrays.

```
$.each(Array, function (index, element))
```

```
$.each(Object, function (index, element))
```

```
var personne= {
  "Nom": "Bond",
  "Prenom": "James"
};
```

```
$.each( personne, function( key, value ) {
  alert( key + ": " + value );
});
```

```
Nom: Bond
Prenom: James
```

LA MÉTHODE ANIMATE()

- La méthode **animate()** effectue une animation personnalisée d'un ensemble de propriétés CSS.
- Cette méthode modifie un élément d'un état à l'autre avec des styles CSS. La valeur de la propriété CSS est changé **progressivement**, pour créer un effet d'animation
- Seules les **valeurs numériques** peuvent être animés (comme **"margin: 30px"**). Les valeurs de chaîne ne peuvent pas être animés (comme **"background-color: red"**).

(selector).animate({styles},[duration], [easing], [callback])

- **[ ] :** Indique un paramètre optionnel
- **Styles :** Un objet contenant des couples propriété/valeur CSS (le style de l'élément à la fin de l'animation)
- **Duration :** un entier (en ms) ou une chaine de caractères (*slow*, *normal*, *fast*).
- **Easing :** une chaine de caractère désignant l'évolution de l'animation (*linear*, *swing*).
- **Callback :** une fonction de retour qui va être appelée lorsque l'animation est finie.

- On peut utiliser **"+ ="** ou **"- ="** pour les animations relatives.
- On peut utiliser la méthode ***stop()*** pour arrêter une animation.

EXEMPLE

- On veut déplacer le carré suivant (id="box") vers la droite de 250px




```
$('#bt').click(function(e) {
        $("#box").animate({left:"250px"},2000);
});
```

EXERCICE 2

```
<button id="stop">Stop</button>

<div id="titre">Clicker pour animer</div>
<div id="contenu">Salut à tous !</div>
```





EXERCICE 3







LE SÉLECTEUR HTML

```
$("<li><a></a></li>")
        .find("a")
                .attr("href", "http://www.google.fr")
                .html("Google")
        .end()
        .appendTo("ul");
```

- Que fait ce code ?